

```
//
=====
=====
// SERVEUR DE L'ORACLE DES ARCANES
// -----
-----
// Ce petit serveur fait 3 choses :
// 1. Crée une "session de paiement"
Stripe quand quelqu'un
//   clique sur "Payer"
(consultation unique ou abonnement)
// 2. Vérifie auprès de Stripe qu'un
paiement a vraiment été
//   payé avant de débloquent l'accès
au site
// 3. Sert votre page du site (le
fichier oracle-des-arcanes.html)
//
=====
=====

const express = require('express');
const Stripe = require('stripe');

// ⚠ ÉTAPE IMPORTANTE : votre clé
secreète Stripe doit être mise
```

```
// dans une "variable
d'environnement" nommée
STRIPE_SECRET_KEY,
// JAMAIS écrite directement dans ce
fichier. On vous expliquera
// comment la configurer sur Render
au moment de l'hébergement.
const stripe =
Stripe(process.env.STRIPE_SECRET_KEY)
;

const app = express();
app.use(express.static('public')); //
sert les fichiers du site (HTML,
etc.)
app.use(express.json());

// L'adresse de votre site une fois
en ligne (à ajuster plus tard)
// Pour l'instant on utilise une
valeur par défaut modifiable.
const SITE_URL = process.env.SITE_URL
|| 'http://localhost:3000';

// -----
-----

// Prix : à ajuster si vous changez
vos tarifs.
// Les montants sont en centimes (290
```

```
= 2,90 €).  
// -----  
-----  
const PRICES = {  
  unite: { amount: 290, label:  
    'Consultation à l\'unité' },  
  abonnement: { amount: 790, label:  
    'Abonnement mensuel' }  
};  
  
// -----  
-----  
// ROUTE 1 : créer une session de  
paiement  
// Le bouton "Payer" du site appelle  
cette route.  
// -----  
-----  
app.post('/creer-paiement', async  
(req, res) => {  
  try {  
    const { offre } = req.body; //  
    "unite" ou "abonnement"  
  
    if (!PRICES[offre]) {  
      return res.status(400).json({  
        erreur: 'Offre inconnue.' });  
    }  
  }  
}
```

```
let session;

if (offre === 'unite') {
  // Paiement unique
  session = await
stripe.checkout.sessions.create({
  mode: 'payment',
  payment_method_types:
['card'],
  line_items: [{
    price_data: {
      currency: 'eur',
      product_data: { name:
PRICES.unite.label },
      unit_amount:
PRICES.unite.amount,
    },
    quantity: 1,
  }],
  success_url: `${SITE_URL}/?
session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${SITE_URL}/?
paiement=annule`,
});
} else {
  // Abonnement mensuel récurrent
  session = await
stripe.checkout.sessions.create({
    mode: 'subscription',
```

```

        payment_method_types:
['card'],
        line_items: [{
            price_data: {
                currency: 'eur',
                product_data: { name:
PRICES.abonnement.label },
                unit_amount:
PRICES.abonnement.amount,
                recurring: { interval:
'month' },
            },
            quantity: 1,
        }],
        success_url: `${SITE_URL}/?
session_id={CHECKOUT_SESSION_ID}`,
        cancel_url: `${SITE_URL}/?
paiement=annule`,
    });
}

// On renvoie l'adresse de
paiement Stripe au site,
// qui va rediriger l'utilisateur
dessus.
res.json({ url: session.url });

} catch (err) {
    console.error('Erreur lors de la

```

```

    création du paiement :',
    err.message);
    res.status(500).json({ erreur:
    'Impossible de créer le paiement.'
    });
  }
});

// -----
// -----
// ROUTE 2 : vérifier qu'un paiement
// a bien été payé
// Le site appelle cette route après
// le retour de Stripe,
// avec l'identifiant de session reçu
// dans l'URL.
// -----
// -----

app.get('/verifier-
paiement/:sessionId', async (req,
res) => {
  try {
    const session = await
stripe.checkout.sessions.retrieve(req
.params.sessionId);

    const estPaye =
session.payment_status === 'paid';

```

```
        res.json({
            paye: estPaye,
            offre: session.mode ===
'subscription' ? 'abonnement' :
'unite',
        });

    } catch (err) {
        console.error('Erreur lors de la
vérification :', err.message);
        res.status(400).json({ paye:
false, erreur: 'Session de paiement
introuvable.' });
    }
});

// -----
-----
// Démarrage du serveur
// -----
-----

const PORT = process.env.PORT ||
3000;
app.listen(PORT, () => {
    console.log(`Serveur de l'Oracle
des Arcanes démarré sur le port
${PORT}`);
});
```

